```
module dev.demo.catalog {
    exports dev.demo.catalog.api;
}
```

Back to the Future with JPMS

Is modular Java actually worth it?



JavaCro'26

Not a JPMS sales pitch

The useful question is not “can we use modules?” but “what problem would modules solve?”



If the result is “not worth it here”, that is still a useful outcome.

A good architecture talk should help even people who decide not to adopt the featured technology.

About me



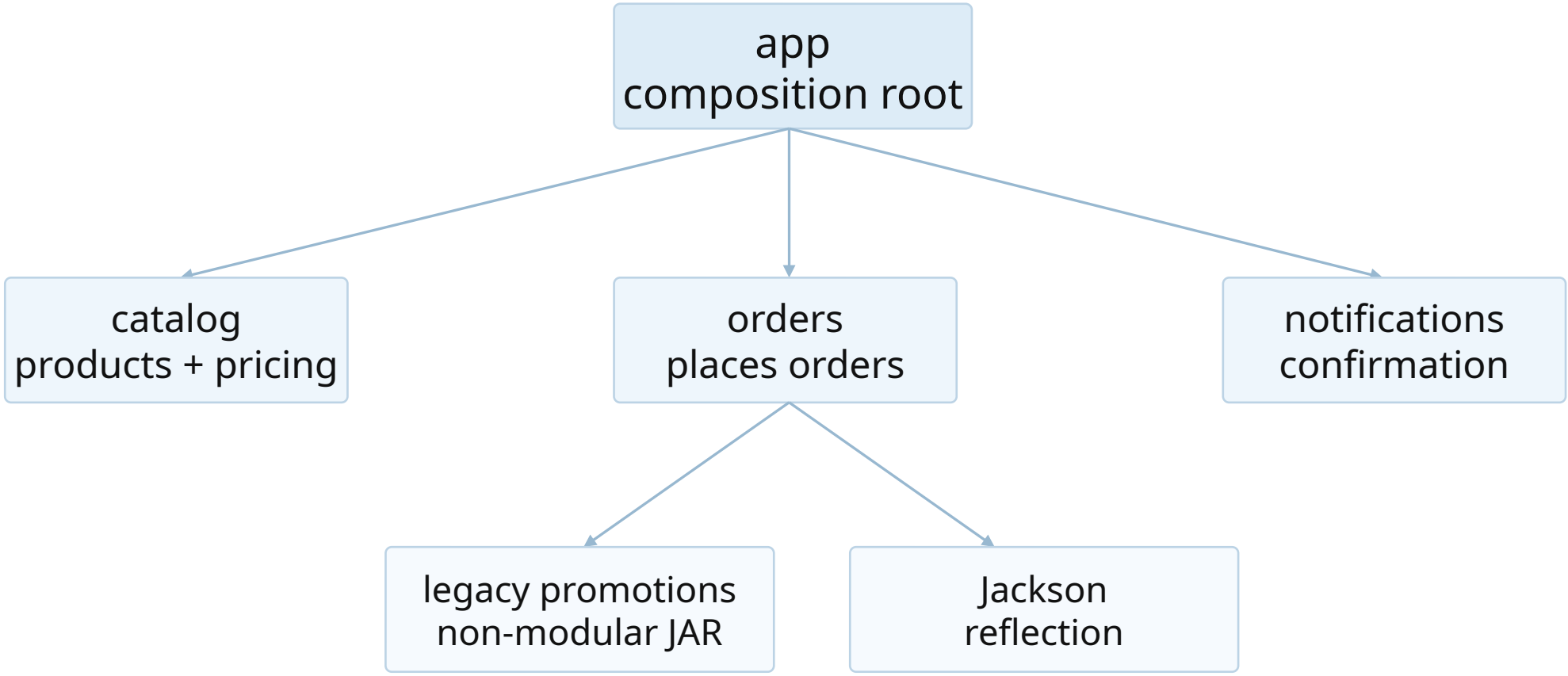
Peter Velychko

- Software Developer
- Java, Kotlin, Scala

- LinkedIn : https://www.linkedin.com/in/petervelychko/

The demo application

Small enough to understand; realistic enough to expose real problems.



Maven modules first, JPMS modules later

Baseline: the classpath works

The first checkpoint deliberately proves that we are not fixing a broken app.

```
java -cp "app/target/app-demo-1.0-
SNAPSHOT.jar:app/target/modules/*"
dev.demo.app.Main
```

```
Java 25 | module: null
Persisted order as JSON: {...}
Notification: order ... placed
```

---

module: null

Everything is in the unnamed module. Nothing prevents accidental cross-boundary access.

JPMS in three verbs

Enough vocabulary to understand the experiment - not a specification tour.

requires

What do I depend on?

exports

What is my API?

opens

Who may reflect into me?

```
module dev.demo.orders {
    requires dev.demo.catalog;
    exports dev.demo.orders.api;
    opens dev.demo.orders.internal.persistence
        to com.fasterxml.jackson.databind;
}
```

First experiment: hide Catalog internals

A small module descriptor turns a convention into a rule.

```
module dev.demo.catalog {
    exports dev.demo.catalog.api;
}
```

dev.demo.catalog.api
Catalog, Product, CustomerTier

dev.demo.catalog.internal
PricingRules, InMemoryCatalog

Only the API package is exported.

The package named “internal” is no longer just a polite suggestion.

The architecture was lying

Orders claimed to depend on Catalog - but it depended on Catalog internals.

```
import dev.demo.catalog.internal.PricingRules;

var price =
 PricingRules.priceFor(product.price(), tier);
```

→

```
[ERROR] package dev.demo.catalog.internal is not
visible
  declared in module dev.demo.catalog,
  which does not export it
```

JPMS did not create the architectural problem.

It discovered it.

This is the strongest reason to care about JPMS: platform-enforced boundaries.

The tempting wrong fix

The compiler can be satisfied without improving the architecture.

```
module dev.demo.catalog {
    exports dev.demo.catalog.api;
    exports dev.demo.catalog.internal;  //
“fixed”
}
```

This compiles.

But the boundary is gone.

Question to ask instead:

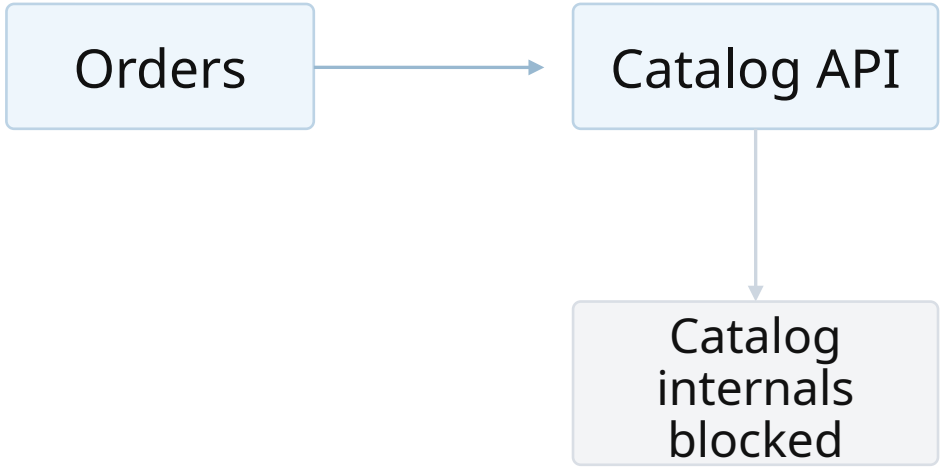
What did we actually intend to expose?

Fix the architecture, not the descriptor

Orders should ask the Catalog API for a price; not borrow its implementation detail.

```
BigDecimal catalogPrice =
    catalog.priceFor(sku, tier);
```

```
public interface Catalog {
    Optional<Product> find(String sku);
    BigDecimal priceFor(String sku, CustomerTier
tier);
}
```



The module descriptor did its job only after we changed the design.

The price of enforcement: reflection

Modern frameworks do not only call public methods.

```
final class StoredOrder {
    private UUID id;
    private BigDecimal total;
}
```

```
java.lang.reflect.InaccessibleObjectException
module dev.demo.orders does not "opens"
dev.demo.orders.internal.persistence
to module com.fasterxml.jackson.databind
```

Compile-time boundary → Runtime reflective access

Strong encapsulation is useful - but frameworks make the cost visible.

`exports` is not `opens`

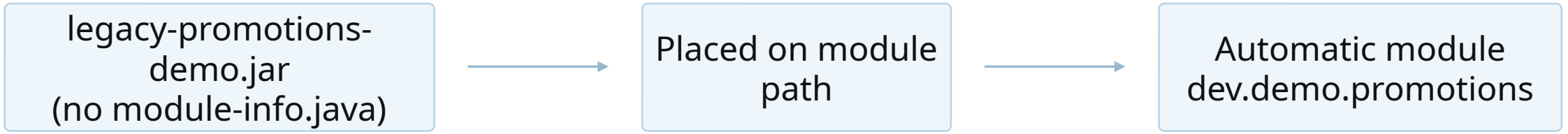
A package can remain non-API while still allowing targeted reflection.

exports	opens	open module
Ordinary Java access compile-time readability	Deep reflection runtime frameworks	Broad reflective opening usually too blunt

```
opens dev.demo.orders.internal.persistence
    to com.fasterxml.jackson.databind;
```

Automatic modules: a migration bridge

Real dependency graphs do not become modular all at once.



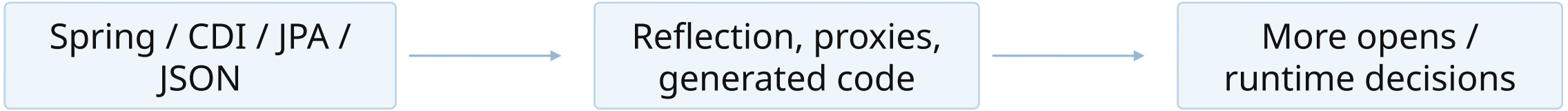

```
Automatic-Module-Name: dev.demo.promotions

requires dev.demo.promotions;
```

Bridge, not destination: automatic modules participate in the graph, but they export broadly.

Framework reality check

Spring, CDI, serialization and proxies move the discussion from “can JPMS work?” to “what is the cost?”



Quarkus is actively exploring this space:

Modularity extension
build-time module graph
experimental

JLink extension
custom runtime image
experimental, Java 25+

Ecosystems are not ignoring modularity - but integration is still a tradeoff.

JPMS is not the only enforcement mechanism

Different tools enforce different kinds of boundaries at different times.

Approach	What it enforces	When	Migration cost
Conventions	human discipline	never automatically	zero
ArchUnit	test/build rule	CI	low
Spring Modulith	Spring application model	test/build + docs	medium
JPMS	Java platform boundary	compile + runtime	higher

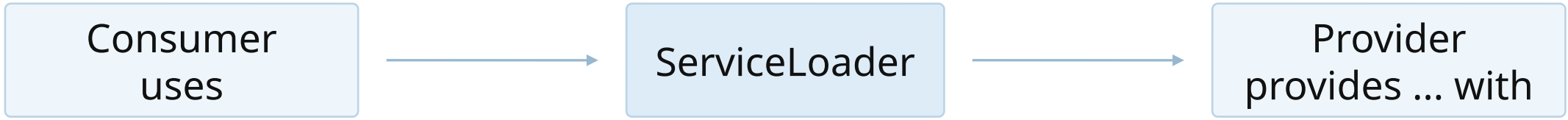
The honest question: do we need architectural verification, or platform enforcement?

Bonus concept: services

A module graph can also support plugin-style discovery.

```
module dev.demo.app {
  uses
    dev.demo.notifications.api.NotificationService;
}
```

```
module dev.demo.notifications.email {
    provides NotificationService
        with EmailNotificationService;
}
```



Useful, but not central to our architecture experiment.

Bonus concept: jlink

A reliable module graph can become a smaller runtime image.




```
jlink --module-path ...
      --add-modules dev.demo.app
      --output runtime-image
```

Good to know, but not the reason to modularize a business application unless distribution size/runtime packaging matters.

Decision framework

Do not modularize because Java has modules. Modularize when enforced boundaries solve a real problem.

Consider JPMS when…

- boundaries are meaningful
- accidental dependencies hurt
- codebase is long-lived
- platform enforcement has value

Consider alternatives when…

- CI verification is enough
- Spring Modulith fits better
- migration cost is high
- framework friction dominates

Avoid JPMS when…

- app is small
- boundaries do not matter
- dependencies fight module path
- cost beats value

Choose the weakest tool that reliably enforces the boundary you care about.

JPMS is not about adding `module-info.java` files.

It is about deciding which architectural boundaries Java should enforce.

Thank you

GitHub Repository: https://github.com/petervelychko/jpms-demo



Questions?